

PeakCheck — User Manual

Interactive multi-peak fitting and peak-presence checking for generic x/y data

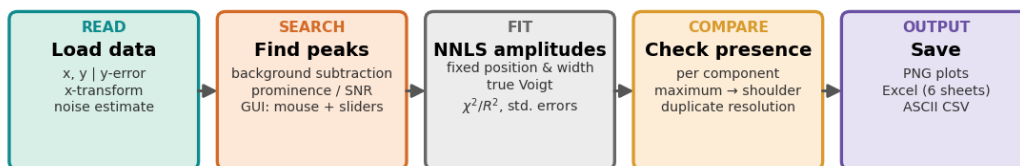
Lukas Knauer

RPTU Kaiserslautern-Landau · AG Schünemann

Version 1.0.0 · MIT license · 2026

What does PeakCheck do? It determines peaks in a reference data set (e.g. a summed spectrum) and then checks whether those same peaks are present in one or more component data sets (e.g. sub-spectra). It works with any 2- or 3-column x/y data — spectra, diffraction patterns, chromatograms, time series.

PeakCheck — stages of an analysis



Reference data set defines the peaks · components are checked against them · one TOML config controls everything

Figure 1. The five stages of an analysis: read, search, fit, compare, output.

1 Installation & start

PeakCheck is a small Python package with a backwards-compatible launcher (`peakcheck.py`). It needs Python 3.10+ and the packages `numpy`, `scipy`, `matplotlib` and `openpyxl`, plus Tk for the graphical interface.

```
# Install the packages (once)
pip install numpy scipy matplotlib openpyxl
# Tk ships with the python.org installer on Windows/macOS; on Linux:
sudo apt install python3-tk
# For Python < 3.11 also the TOML reader:
pip install tomli
```

There are three equivalent ways to start it. Pick whichever fits your habits:

```
# 1) Graphical (default): open a window, tune, click, fit & save
python -m peakcheck          # or simply: peakcheck (after pip install)
python peakcheck.py          # still works as a backwards-compatible launcher

# 2) Headless, driven by a TOML file (batch / HPC)
python -m peakcheck --config job.toml --no-gui

# 3) Write a commented configuration template
python -m peakcheck --write-template job.toml
```

In Spyder, run it as a standalone program (not inside the IPython console) so a real window opens; or set `%matplotlib qt5` in the console beforehand.

2 Input data

PeakCheck reads plain-text column data regardless of the file extension: `.dat`, `.txt`, `.csv`, `.xy`, `.asc`, `.tsv`, `.prn`, `.nis` and similar all work, as long as the file contains two or three numeric columns. Values may be separated by whitespace, tabs, commas or semicolons; header lines, comment characters (`## ! \%`) and a German-style decimal comma (e.g. `10,5`) are detected and handled automatically. (The one extension treated specially is `.fio`; see below.)

Format	Meaning	Effect on the fit
x y	two columns, no errors	unweighted fit; noise is estimated from the data
x y y_error	three columns with a measurement error	weighted fit; errors enter as $1/\sigma$

FIO files (DESY/PETRA III). Files ending in `.fio` are read directly: PeakCheck parses the `\%d` data block with its `Col` column declarations (the `\%c/\%p` comment and parameter blocks are ignored). Because such files hold many columns, two settings choose which to use: `fio_x_column` (default: the first column, typically the energy axis) and `fio_y_column` (default `"nisp"`, the NIS signal). Either may be a column *name* (e.g. `"nfsp"`) or a 1-based index. FIO files carry no per-point error column, so the noise is estimated unless you point `error_column` at a specific column. reference is `sample.txt`, then `sample_01.txt`, `sample_42.txt` etc. are picked up as components. The pattern is configurable via `component_glob`.

3 The graphical interface

Workflow

1. **Open a folder** via *Open folder...* A dialog lists every data file in the folder. Pick one as the reference (radio button in the “Ref” column) and any number as components (check boxes in the “Cmp” column). Confirm with *OK* — the data appear at once together with the background-corrected curve (blue).
2. **Restrict the search range** with `x_min/x_max` (confirm with *Enter*); the active range is shaded orange.
3. **Choose profile and widths**: `voigt`, `gauss` or `lorentz`, plus the FWHMs `WG` (Gaussian) and `WL` (Lorentzian). The resulting effective FWHM of the chosen profile is shown next to them as a read-only value (it updates as you edit `WG/WL` or the profile).
4. **Find peaks** with the four sliders (prominence, minimum distance, minimum SNR, background window) controlling the automatic search; what each one does is explained under “Choosing parameters” below.
5. **Baseline and refinement** (fourth row, optional): choose the `baseline_method` used by the fit (`rolling_min`, `polynomial` or `als`, with the polynomial order and ALS λ/p next to it), and tick *Refine positions* to refine the centres within `refine window` after the fit. The defaults reproduce the previous behaviour.
6. **Refine by mouse** directly in the plot:
 - *Left click* — add a peak at the nearest data point
 - *Right click* — remove the nearest peak marker
7. **Fit & Save** runs the fit, the presence check across components, and writes all outputs.

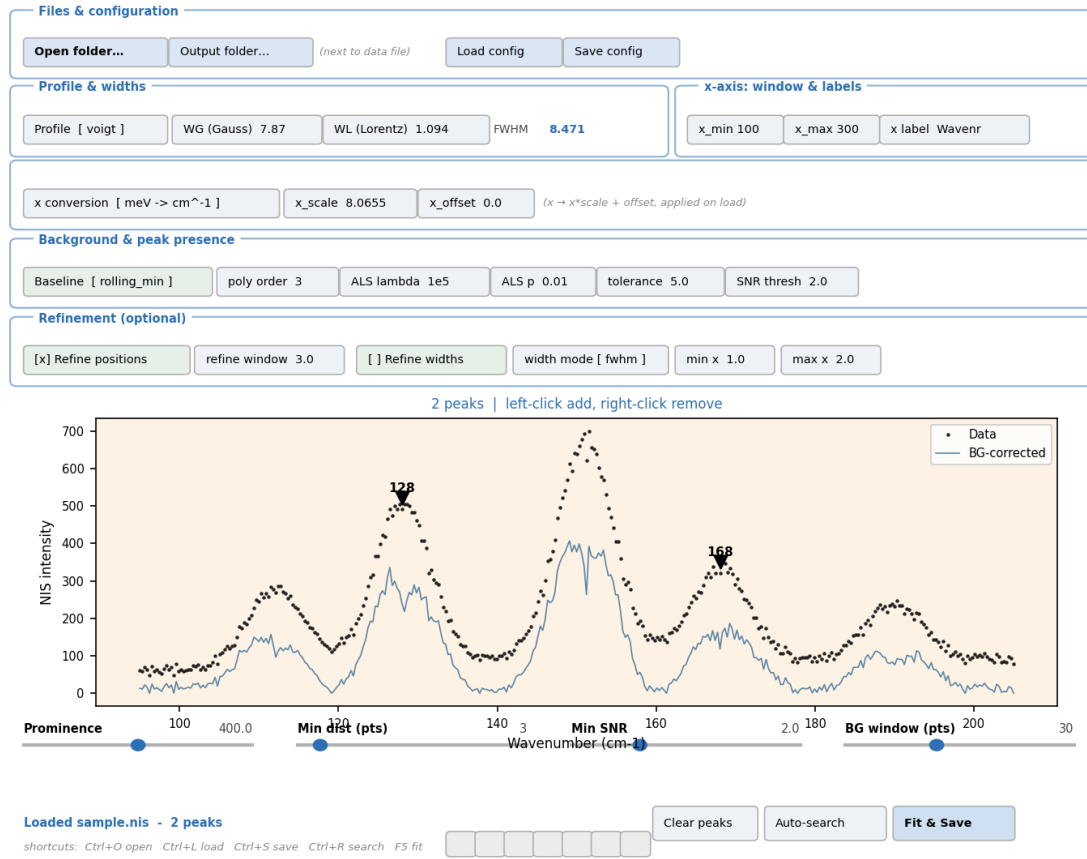


Figure 2. The PeakCheck window: the controls are grouped into labelled frames (Files & configuration; Profile & widths; x-axis window, labels & conversion; Background & peak presence; Refinement), followed by the interactive plot with peak markers, the search sliders, and the *Fit & Save* button. Keyboard shortcuts are shown along the bottom. Fields that do not apply to the current selection (e.g. the ALS parameters when the baseline is not ALS) are greyed out.

Suggested order: set the search range first, then adjust the background window so the blue curve traces the peaks cleanly, then fine-tune minimum SNR (≈ 2) and prominence — and finally add or remove individual peaks by mouse.

Via *Save config* / *Load config* all settings can be stored to and reloaded from a TOML file, making every analysis reproducible.

3.1 Converting the x-axis units

The third control row converts the x-axis on load. The *x conversion* drop-down offers common conversions as presets; it sets **x_scale**, **x_offset** and the axis label/unit automatically. For NIS/NRVS pick “meV $\rightarrow$ cm⁻¹” — the whole analysis then runs in cm⁻¹.

The GUI presets are the linear (affine) conversions $x \rightarrow x \cdot \mathbf{x_scale} + \mathbf{x_offset}$; you can also enter a custom factor in **x_scale**/**x_offset**. Reciprocal conversions (wavelength $\leftrightarrow$ wavenumber/energy, e.g. cm⁻¹ = 10⁷/λ [nm]) are non-linear and are available through the TOML file only (see §4).

Important: the conversion is applied on load. All other values given in x units — **x_min**, **x_max**, **wg**, **wl**, **tolerance** — therefore refer to the already converted values (e.g. cm⁻¹, not meV).

Preset	factor (x_scale)	typical use
meV $\rightarrow$ cm ⁻¹	8.065544	NIS/NRVS, pDOS
cm ⁻¹ $\rightarrow$ meV	0.12398419	reverse
meV $\rightarrow$ THz	0.24179893	THz / inelastic spectroscopy
cm ⁻¹ $\rightarrow$ THz	0.029979246	FIR, phonons
eV $\rightarrow$ cm ⁻¹	8065.544	general energy axes
meV $\rightarrow$ K (k_B)	11.604518	thermal scale

4 Command line

Besides the GUI, PeakCheck has a small command line (the `peakcheck` console script, equivalently `python -m peakcheck`):

Command	Effect
<code>peakcheck</code>	launch the graphical interface
<code>peakcheck -write-template job.toml</code>	write a commented config template and exit
<code>peakcheck -config job.toml -no-gui</code>	headless (batch) run; needs a config
<code>peakcheck -config job.toml -validate</code>	check a config and exit (non-zero exit on error)
<code>peakcheck -list-conversions</code>	list the x-axis presets and exit
<code>peakcheck -reference data.dat</code>	override the reference file from the config
<code>peakcheck -output-dir DIR</code>	write all outputs to DIR (batch; overrides <code>output_dir</code>)
<code>peakcheck -version</code>	print the version
<code>peakcheck -debug</code>	on error, show the full traceback instead of a short message

A headless run writes exactly the same plots, Excel workbook and CSV as the GUI, so a config saved from the GUI reproduces its result in batch. Use `-validate` in scripts to fail fast on a malformed config. On a bad config or data file the command prints a short **PeakCheck:** message and exits non-zero; add `-debug` to see the full Python traceback.

5 A first analysis

The package ships a ready-to-run example in `example_data/`.

Headless, in one line (from `example_data/`):

```
python -m peakcheck --config nis_example.toml --no-gui
```

PeakCheck loads the reference `sample.nis`, fits five peaks, checks the two component files `sample_A.nis` and `sample_B.nis`, and writes the plots, the Excel workbook and the CSV next to the data.

Interactive: start `peakcheck`, click *Open folder...* (Ctrl+O) and pick `sample.nis` as the reference, `sample_A.nis` and `sample_B.nis` as components. Choose the *x conversion* `meV -> cm-1`, press *Auto-search peaks* (Ctrl+R) or click the peaks by hand, then *Fit & Save* (F5); see Figure 2.

The fit reproduces the amplitudes, $\chi^2_\nu \approx 5.5$ and $R^2 \approx 0.945$ from the supplement and reports which of the five peaks each component contains.

6 Configuration via TOML

For batch runs and reproducible pipelines a TOML file controls every parameter. A commented template is produced by `python -m peakcheck -write-template job.toml`. The reference below

lists all available keys grouped by TOML section, with their default value, meaning and a hint when to change them.

Key	Default	Meaning
reference_file	"data.txt"	Path to the reference file whose peaks are determined first. A relative path is resolved against the config file's own directory, so a config and its data stay portable (batch-friendly); absolute paths are used as given. A command-line -reference overrides this and keeps the usual shell (working-directory) semantics.
component_glob	"" (empty)	Glob pattern for component files. Empty = automatic <code><stem>_*.<ext></code> (e.g. <code>sample.nis</code> → <code>sample_A.nis</code>). Override with an explicit pattern (e.g. <code>"sub_*.dat"</code>); from the GUI the selection is passed in directly via the folder dialog.
error_column	"auto"	"auto" = use the third column as the error if present, else estimate from data. "none" = always estimate. Integer = explicit 0-based column index.
fio_x_column	""	FIO files only: x column by name or 1-based index; empty = first column.
fio_y_column	"nisp"	FIO files only: y column by name (e.g. "nfsp") or 1-based index.

[input]

Key	Default	Meaning
x_conversion	"" (empty)	Named preset such as "meV -> cm ⁻¹ ", "cm ⁻¹ -> THz" or "nm -> eV (reciprocal)". Sets scale, offset, label and unit automatically. The TOML template lists all presets.
x_transform	"affine"	"affine": $x' = x \cdot s + o$ (most energy units). "reciprocal": $x' = s/x + o$ (e.g. wavelength → wavenumber).
x_scale, x_offset	1.0, 0.0	Manual scale factor and offset. Ignored when x_conversion is set.
x_label, y_label	"x", "y"	Axis labels used in plots and result files.
x_unit	"" (empty)	x-axis unit; shown in parentheses behind the label and in the Excel unit header row (Origin style).

[xaxis]

Key	Default	Meaning
profile	"voigt"	"voigt" (exact Gaussian⊗Lorentzian convolution), "gauss", "lorentz". Voigt is the physically correct default for most spectroscopies.
wg	7.8701	Gaussian FWHM in x-units — the instrumental resolution. For NIS at BL09XU (SPRING-8): ~ 0.8 meV ≈ 6.4 cm ⁻¹ ; at P01 (PETRA III): see beamline spec.
wl	1.094	Lorentzian FWHM in x-units — natural linewidth plus thermal contributions. For non-decaying phonons usually small compared to wg.

[profile] — line shape and widths

[window] — search and fit window

[search] — automatic peak search

Key	Default	Meaning
<code>x_min, x_max</code>	<code>"", ""</code>	Lower and upper bound of the analysis window in x-units. Empty = open (full file). Influences peak search and fit; appears in the output filename for traceability (e.g. <code>_100_200</code>).
<code>bg_window</code>	<code>30</code>	Window size for background subtraction (rolling minimum, in points). Rule of thumb: somewhat wider than a peak, much narrower than the baseline curvature. <code>0</code> disables it.
<code>baseline_method</code>	<code>rolling_min</code>	Baseline estimator used by the fit, statistics and presence check: <code>rolling_min</code> (default), <code>polynomial</code> or <code>als</code> (asymmetric least squares). The smooth alternatives suit curved baselines; <code>baseline_poly_order</code> (<code>3</code>), <code>baseline_als_lambda</code> (10^5) and <code>baseline_als_p</code> (<code>0.01</code>) tune them.

Key	Default	Meaning
<code>prominence_init</code>	<code>200.0</code>	Minimum prominence in y-units (<code>scipy.signal.find_peaks</code>). Higher values reject noise but miss weak peaks. Live-adjustable via slider in the GUI.
<code>distance_init</code>	<code>3</code>	Minimum distance between peaks in points. Prevents adjacent noise spikes from being counted as two peaks.
<code>min_snr_init</code>	<code>2.0</code>	SNR threshold for the auto-search (background-corrected signal). Accepts a peak only if its height $\geq$ <code>min_snr_init</code> .error.

[refine] — optional position and width refinement Set `refine = true` to refine the peak positions after the fit by a bounded nonlinear step (the amplitudes stay non-negative); `refine_window` (default `3.0`, in x-units) limits how far each position may move. Set `refine_widths = true` to additionally refine one width per peak: `width_mode` chooses what broadens ("`fwhm`", the default, scales the whole Voigt; "`sigma`" scales only the Gaussian part), and the width is kept within `width_min_factor`..`width_max_factor` times the instrumental width (defaults `1.0`..`2.0`, i.e. never narrower than the instrument and at most twice as wide). Both are off by default, so the result is unchanged unless you enable them.

Key	Default	Meaning
<code>tolerance</code>	<code>5.0</code>	$\pm$ position window in x-units within which a component peak counts as "the same" as a reference peak. Should be larger than the typical centroid shift between spectra, smaller than the peak spacing.
<code>snr_thresh</code>	<code>2.0</code>	SNR threshold above which a peak in the component counts as "present". Raise (<code>4–6</code>) for noisy spectra, lower (<code>2–3</code>) for clean spectra.
<code>presence_baseline_corrected</code>	<code>true</code>	<code>true</code> = SNR measured on the background-corrected signal (height above baseline — the physically meaningful value). <code>false</code> = SNR on the raw data (includes background); rarely wanted.

[presence] — presence check in components

[errors] — error reporting

[output] In addition the peak positions can be fixed directly instead of picking them interactively or trusting the auto-search:

Key	Default	Meaning
error_mode	"statistical"	"statistical" = true 1σ from $(B^T W^2 B)^{-1}$, unscaled (matches <code>absolute_sigma=True</code> in <code>scipy.curve_fit</code>). "scaled" = multiplied by $\sqrt{\chi_{\text{red}}^2}$ — appropriate when only relative error weights are known.

Key	Default	Meaning
output_dir	"" (empty)	Target folder for all outputs (plots, Excel, CSV). Empty = next to the reference file. In the GUI also selectable via the "Output folder..." button.
write_csv	true	Write the ASCII CSV with all result sections. Setting to false keeps only the Excel file.
write_excel	true	Write the Excel workbook.
plot_dpi	150	Resolution of the saved PNG plots. 150 is fine for manuscript previews; raise to 300+ for print.

```
peaks = [112.0, 128.0, 151.0, 168.0, 190.0]
```

The list is taken as the final set of peaks — the auto-search is skipped entirely. This is the most reproducible mode for manuscript pipelines.

7 Choosing parameters & good practice

A short, practical guide; the supplement derives the reasoning.

Widths (wg, wl). These are the *instrumental* resolution and are held fixed. Set them from the instrument/beamline, not from the data. A reduced $\chi_\nu^2 \gg 1$ usually means the widths do not match the data (or a peak is missing), not that the amplitudes are wrong.

Background (baseline_method). The default rolling minimum is robust for narrow peaks on a broad pedestal. For a smoothly curved background the **polynomial** or **als** methods follow the curve more faithfully; ALS is usually the best general choice (Figure 3). The same baseline is used by the fit, the statistics and the presence check.

The four search sliders. The automatic search (`scipy.signal.find_peaks` on the background-corrected signal) is driven by four controls, all live in the GUI:

- **Prominence** (`prominence_init`, y-units): how far a peak must stand out from its local surroundings — the height of the summit above the higher of its two neighbouring valleys, not the absolute y-value. This is the main control against noise: raise it to reject noise spikes, lower it to catch weak bands.
- **Minimum distance** (`distance_init`, in data points): the smallest allowed separation between two detected peaks. It stops a noisy flank from being counted as several closely spaced peaks; when two candidates are closer than this, the more prominent one wins.
- **Minimum SNR** (`min_snr_init`): a candidate is accepted only if its height is at least `min_snr_init` times the estimated noise. This complements prominence with a noise-relative criterion.
- **Background window** (`bg_window`, in points): the window width of the background estimate (rolling minimum, or the chosen **baseline_method**) that is subtracted *before* the search. Too

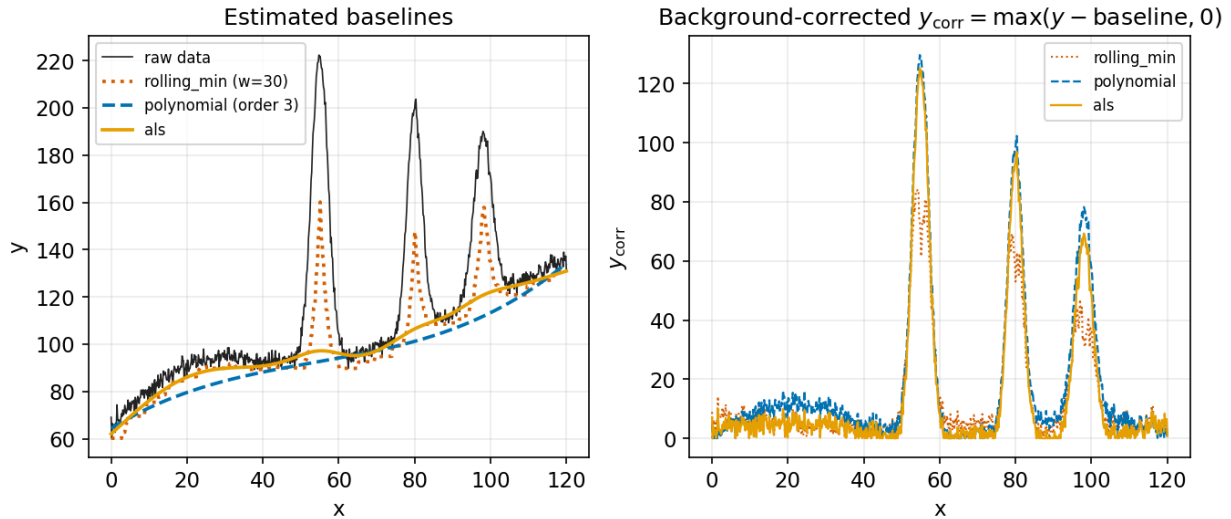


Figure 3. The three baseline methods on one curved, noisy background. Left: the estimated baselines — the rolling minimum tracks the lower envelope in steps, while the polynomial and ALS curves are smooth. Right: the corrected signal; ALS and the polynomial flatten the background between the peaks more cleanly.

small eats into real peaks; too large leaves a curved baseline standing; 0 turns subtraction off. The same background feeds the fit and the presence check.

In the GUI you can also just click the peaks (left-click to add, right-click to remove), which is often faster than fine-tuning the sliders.

Tolerance (`tolerance`). The $\pm$ window in which a component peak counts as “the same” as a reference peak. Make it larger than the typical centroid shift between spectra but smaller than the peak spacing.

Position refinement (`refine`). Off by default. Enable it when the positions are slightly off (e.g. a small calibration shift): the centres are then refined within `refine_window` while the amplitudes stay non-negative (Figure 4). Leave it off when the positions are known and trusted.

Width refinement (`refine_widths`). Off by default; the widths are otherwise held at the instrumental resolution for all peaks. Enable it when a band is genuinely broader than the instrument (e.g. unresolved splitting or inhomogeneous broadening). One width per peak is then refined, bounded between `width_min_factor` and `width_max_factor` times the instrumental width; `width_mode` selects whether the whole Voigt (“`fwhm`”) or only the Gaussian part (“`sigma`”) broadens (Figure 5). Keep the upper bound tight: extra free widths always lower the reduced χ^2_ν and, for overlapping or weak peaks, width and amplitude trade off, so a smaller χ^2_ν is not by itself evidence of a better model.

8 Output files

All results are written next to the reference file; an active search range appears in the file name (e.g. `_90_200`).

The **Presence** sheet is the key result: a binary matrix showing 1 (green, present) or 0 (red, absent) for each peak and component. The **Parameters** sheet documents every run in full, keeping it reproducible.

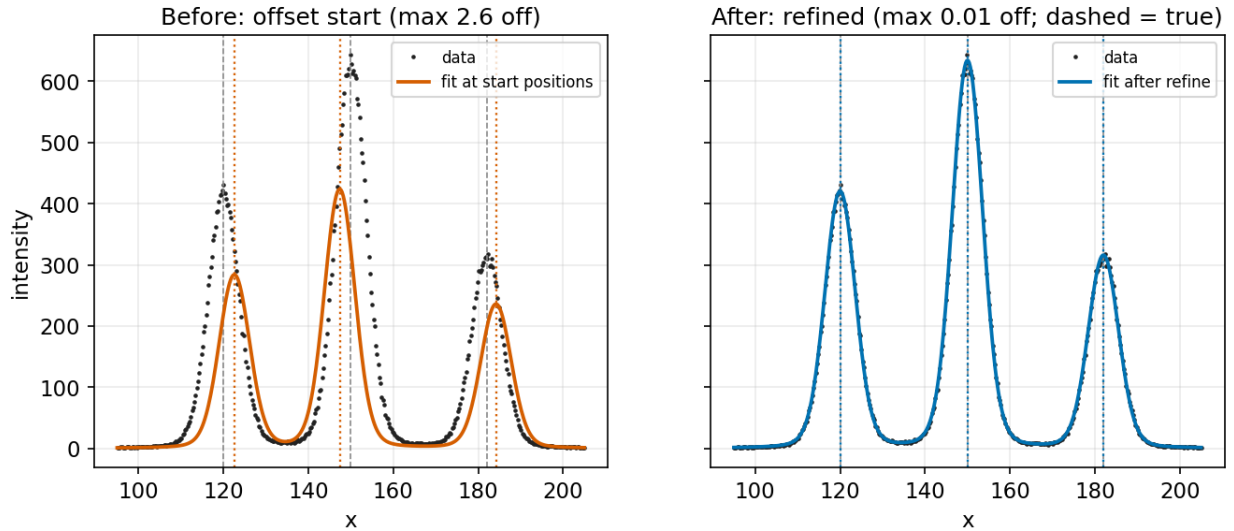


Figure 4. Optional position refinement. Left: a fit with deliberately offset start positions (dotted) misses the true centres (dashed grey). Right: with `refine = true` the centres are recovered to a small fraction of a point.

File	Content
*_fit.png	fit with individual components and a residual panel
*_peaks.png	one marked plot per component (green = present, red = absent)
*_results.xlsx	Excel with six sheets: Intensities, Positions, Presence, Fit_Data, Components_Raw, Parameters (Components_Raw only when components are analysed)
*_results.csv	a single plain 7-bit ASCII CSV with five labelled sections (Intensities, Positions, Presence, Fit_Data, Components_Raw) and the full parameter block as a header

Reading the plots. Figures 6 and 7 label the two plot types. In the fit plot the black curve is the sum, the coloured curves are the individual peaks (each area is the fitted amplitude), and the lower panel shows the residuals — a good fit leaves them scattered around zero with no structure, and the title reports χ^2_ν , R^2 and how many peaks ended up active. In a component plot a green dashed line with a marker means the peak is present (1); a red dashed line means it is absent (0). The decision is taken within the $\pm$ `tolerance` window (shaded) using the SNR threshold.

9 Troubleshooting & FAQ

The GUI does not start. On a headless machine (no display) or without Tk, use the command line: `peakcheck -config job.toml -no-gui`. On Debian/Ubuntu install Tk with `sudo apt install python3-tk`.

“-no-gui needs a config file”. Headless mode needs a TOML (or at least `-reference`). Write one with `peakcheck -write-template job.toml`.

No peaks are found. Lower `prominence_init` and/or `min_snr_init`, check the search window `x_min/x_max`, or simply click the peaks in the GUI. Confirm the *x conversion* so positions are in the units you expect.

Reduced χ^2_ν is far above 1. The fixed-shape model is being strained: the widths `wg/wl` probably do not match the data, or a peak is missing. Add the missing peak or correct the widths; the amplitudes remain the best non-negative fit.

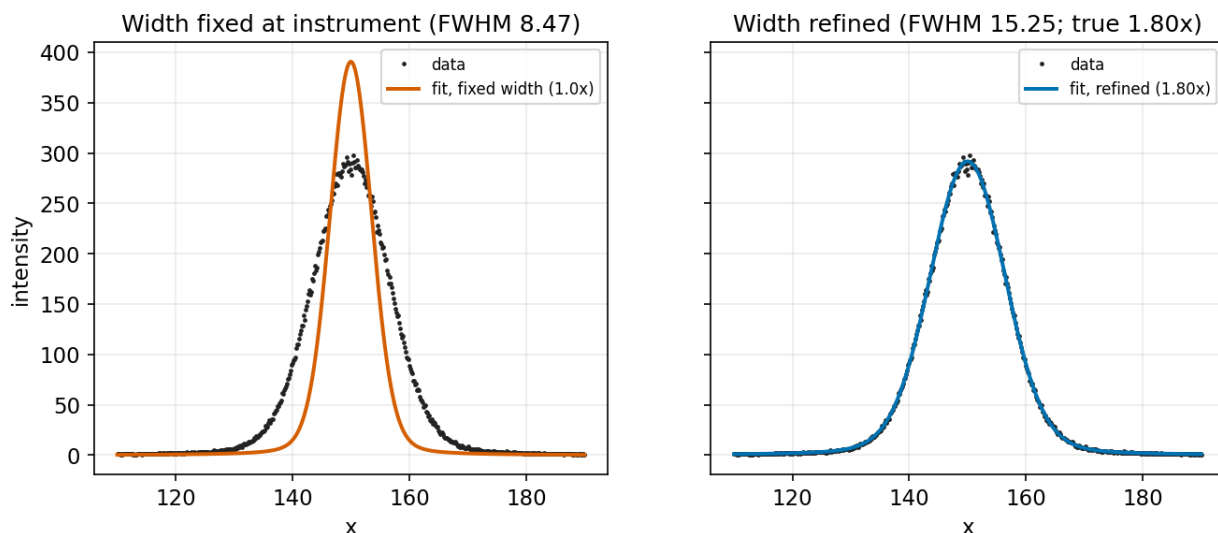


Figure 5. Optional per-peak width refinement on a peak that is genuinely broader than the instrument. Left: with the width fixed at the instrumental resolution the fit cannot reach the data. Right: with `refine_widths = true` the single per-peak width is recovered (here a factor close to the true broadening), bounded by `width_min_factor..width_max_factor`.

Peaks I placed disappear after the fit (active count drops), or the fit is one broad hump.

The profile is far wider than the data’s peaks, so the overlapping profiles become redundant and the non-negative fit sets some amplitudes to zero. Almost always the widths `wg/wl` are on the wrong scale: e.g. they are instrumental values in cm^{-1} but the data are still in meV because the *x conversion* is off. Set the conversion (or load the matching config), or set `wg/wl` to roughly your peaks’ half-width. PeakCheck prints a hint in this case. The marker labels and the fit labels always show the same positions; only their displayed precision matches now (one decimal), so a peak does not “move” between picking and fitting.

A peak I expect is reported “absent”. Raise `tolerance` if the component peak is shifted, or lower `snr_thresh` for a noisy component. Check that `presence_baseline_corrected` matches your intent (height above baseline vs. raw signal).

The reported errors look wrong. With a real error column the `statistical` mode gives true 1σ ; without one, errors are scaled by $\sqrt{\chi^2_\nu}$. Supply an error column for count data spanning orders of magnitude. The supplement shows a Monte-Carlo check of the error budget.

The file will not load. PeakCheck reads two- or three-column whitespace or CSV text and ignores comment lines beginning with `#`, `!` or `%`; non-finite rows are dropped. Check the delimiter and the error-column setting.

Can I reproduce a run exactly? Yes — save the config (*Save config*, Ctrl+S) or reuse the parameter block from the CSV header / Excel **Parameters** sheet, and rerun with `-config`.

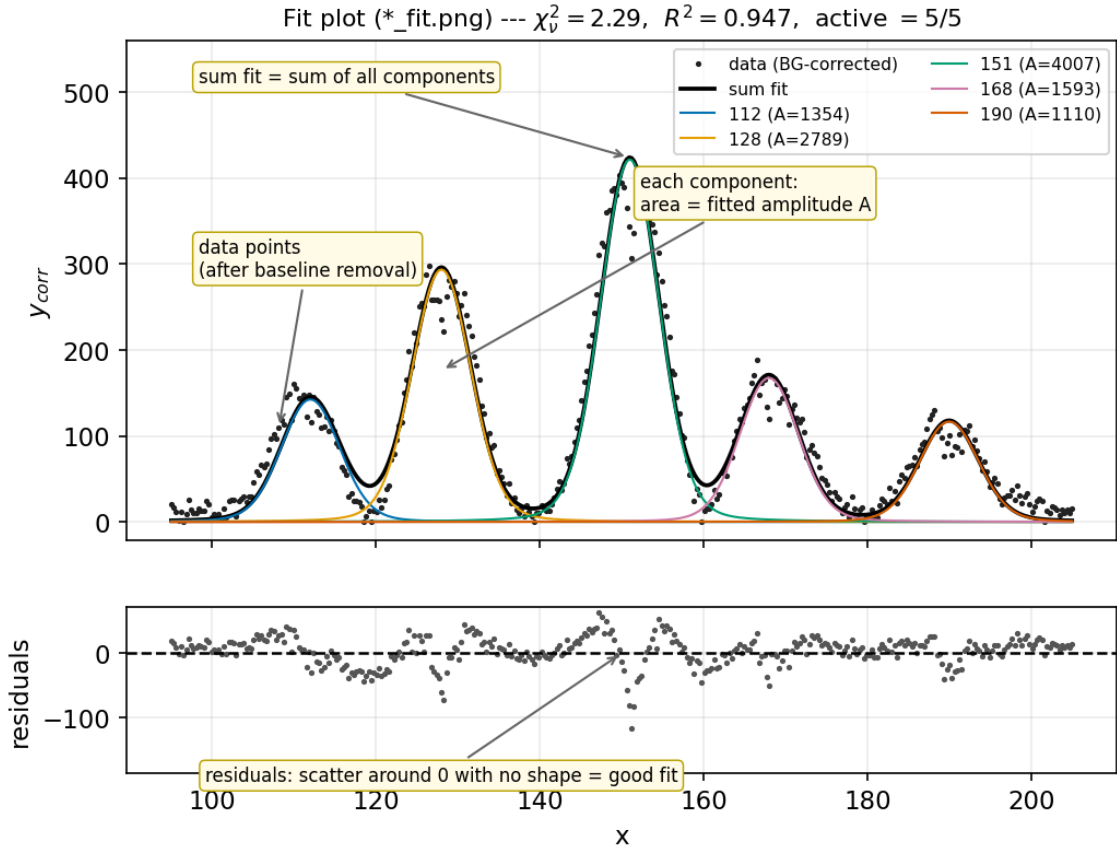


Figure 6. How to read *_fit.png: the data after baseline removal, the summed fit, the individual components (area = amplitude), the goodness-of-fit summary in the title, and the residual panel below.

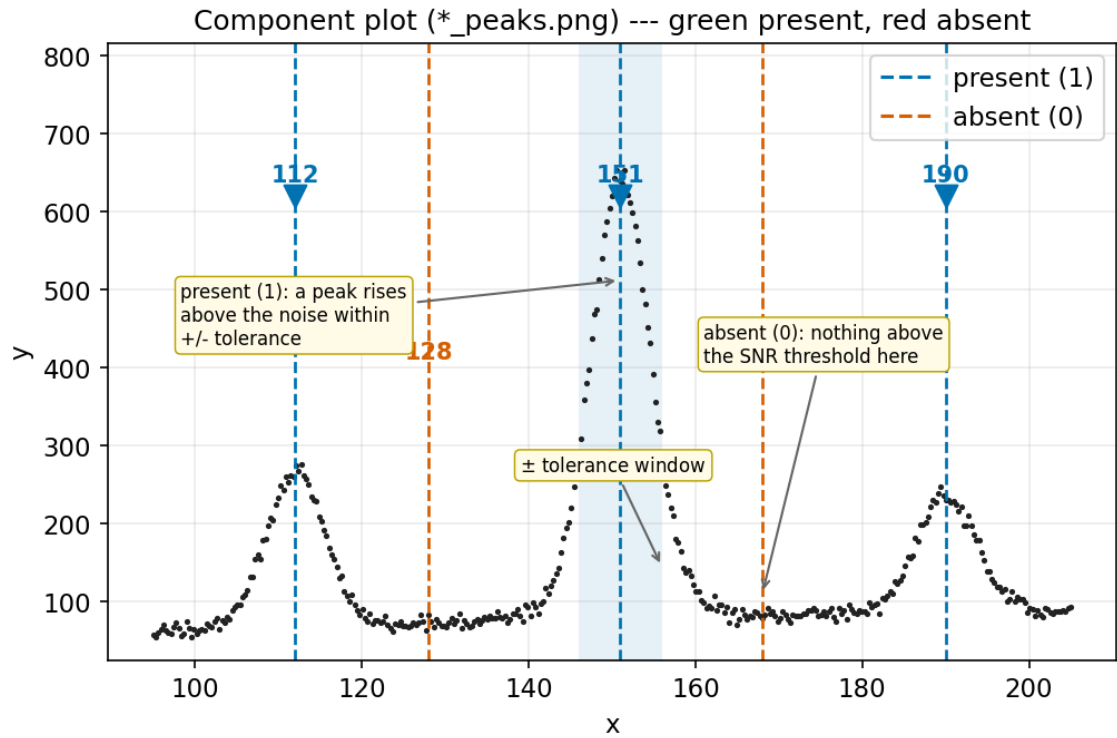


Figure 7. How to read *_peaks.png: a green marker/line flags a peak that is present (1), a red line one that is absent (0); the call is made within the $\pm$ tolerance window (shaded).